

[260620~21] 작업 진행현황(박성은)

☼ 상태	완료
☰ 태그	
🕒 생성 일시	June 20, 2026 4:37 PM

대댓글 기능 구현

변경 내용:

- Comment 모델에 parent 자기참조 필드 추가
 - 최상위 댓글: parent = null
 - 대댓글: parent = 댓글 id
- 댓글 목록 API가 최상위 댓글만 내려주고, 각 댓글 안에 replies 배열로 대댓글을 포함
- 대댓글은 1단계까지만 허용
 - 대댓글에 또 대댓글을 다는 구조는 막아두었습니다.
- 부모 댓글과 다른 게시글에 대댓글을 다는 것도 validation으로 차단
- 프론트 댓글 목록에서 답글 버튼 활성화
- 답글 입력 폼 표시 및 등록 API 연결
- 대댓글에도 좋아요 버튼 표시
- mock 게시글에서도 답글 작성이 화면상 동작하도록 처리

수정 파일:

- back/community/models.py
- back/community/serializers.py
- back/community/views.py
- back/community/tests.py

- back/community/migrations/0004_comment_parent.py
- front/lumiere-frontend/src/services/communityApi.js
- front/lumiere-frontend/src/components/community/CommentThread.vue
- front/lumiere-frontend/src/components/community/CommunityDetailContent.vue
- front/lumiere-frontend/src/views/community/CommunityDetailView.vue

검증:

- python manage.py migrate community 적용 완료
- 대댓글 API 직접 호출 확인: 201 Created
- 댓글 목록에서 replies 배열 반환 확인
- python manage.py test community 통과, 6 tests OK
- npm run build 통과.

진단받은 퍼스널 컬러로 프로필 넣어주기

▼ 방법론

1. 이미지들은 어느 DB에 넣어야 할까요?

결론부터 말씀드리면, 이 이미지들은 **데이터베이스(DB)**에 넣지 않습니다.

유저가 직접 스마트폰으로 찍어서 올리는 셀카 사진은 백엔드(Django)의 DB와 AWS S3 같은 클라우드 스토리지에 저장하는 것이 맞습니다. 하지만 지금 준비하신 32개의 아이콘은 서비스가 기본적으로 제공하는 '시스템 고정 디자인 요소(Static Asset)'입니다.

따라서 백엔드 DB에 넣을 필요 없이, **프론트엔드(Vue) 프로젝트 폴더 안에 직접 넣어두는** 것이 가장 속도도 빠르고 정답에 가까운 설계입니다.

- **추천 경로:** Vue 프로젝트 폴더 안의 `public/images/tones/` 폴더를 하나 만들어서 올려 주신 32개의 `.png` 파일들을 몽땅 집어넣으시면 됩니다.

2. 백엔드(Django)의 역할: "가장 최근 진단 결과 알려주기"

백엔드는 이미지를 줄 필요가 없습니다. 그저 유저가 로그인하면, 진단 기록 테이블(DB)을 시간순으로 정렬해서 **가장 최근에 진단받은 텍스트 데이터 하나만** 프론트엔드로 던져주면

됩니다.

- **Django의 로직 예시:** "수빈 유저의 진단 기록을 생성일(`created_at`) 내림차순으로 정렬해서 가장 첫 번째 결과(`여름 쿨 라이트`)를 주세요!"

💡 3. 프론트엔드(Vue)의 역할: "텍스트에 맞춰 이미지 짝지어주기"

이제 Vue에서는 백엔드가 준 `여름 쿨 라이트` 라는 글자를 보고, 아까 `public` 폴더에 넣어둔 `summer_cool_light.png` 파일을 화면에 띄워주기만 하면 됩니다.

올려주신 5번째 캡처 화면의 파일명들을 바탕으로, 텍스트와 파일명을 연결해 주는 **매핑 사전(Dictionary)** 코드를 짜드릴게요. (작업하시는 아무 Vue 파일의 `<script setup>` 에 넣고 쓰시면 됩니다)

구현 내용:

- 이미지 파일은 DB에 넣지 않고 `public/images/tones/` 경로를 그대로 사용하도록 연결했습니다.
- 백엔드에 "가장 최근 진단 결과" API를 추가했습니다.
 - `GET /api/diagnosis/latest/`
 - 로그인한 사용자의 최신 진단 결과를 반환합니다.
- 백엔드에 사용자별 진단 결과 모델을 추가했습니다.
 - `DiagnosisResult`
 - `user`
 - `personal_color`
 - `confidence_score`
 - `created_at`
- 프론트에 32개 톤 이미지 매핑 파일을 추가했습니다.
 - 실제 파일명 기준으로 매핑했습니다.
 - `fall_*`
 - `spring_warm_*`
 - `summer_warm_ligth.png`
같은 현재 파일명 오타도 흡수합니다.

- 마이페이지 프로필 이미지가 최근 진단 결과의 퍼스널컬러 이미지로 표시되도록 연결했습니다.
- 진단 결과 페이지의 프로필 이미지 박스도 톤 이미지로 표시되도록 바꿨습니다.

수정/추가 파일:

- back/diagnosis/models.py
- back/diagnosis/serializers.py
- back/diagnosis/views.py
- back/diagnosis/urls.py
- back/diagnosis/migrations/0002_diagnosisresult.py
- back/Lumiere/urls.py
- front/lumiere-frontend/src/data/tonelImages.js
- front/lumiere-frontend/src/services/diagnosisApi.js
- front/lumiere-frontend/src/views/accounts/MyPageView.vue
- front/lumiere-frontend/src/views/diagnosis/PersonalColorResultView.vue

검증:

- python manage.py migrate diagnosis 완료
- GET /api/diagnosis/latest/ 직접 호출 확인: 200
- 이미지 파일 존재 확인: summer_cool_light.png, spring_warm_light.png, summer_warm_ligth.png
- npm run build 통과
- python manage.py test community 통과

고급스러운 디자인 수정하기

▼ 방법론

 고급스러운 UI를 만드는 3가지 비밀

1. 배경은 한 톤 '다운(Down)', 콘텐츠는 순백색 '업(Up)'

목업을 보시면 바닥에 깔린 전체 배경색은 순백색이 아닙니다. 약간의 스킨톤이 도는 따뜻하고 차분한 웜 베이지(#faf5f2 등)가 깔려 있습니다. 그리고 그 위에 올라간 사진 업로드 박스나 진단 결과 박스들이 완벽한 순백색(#ffffff)입니다. 이 명도 차이가 화면에 '입체감'과 '고급스러움'을 만들어냅니다.

2. 섬세한 테두리(Border)와 은은한 그림자(Shadow)

순백색 카드들이 배경과 분리되어 동동 떠다니지 않도록, 아주 연한 핑크베이지 색상의 얇은 선(1px solid)이 카드를 감싸고 있습니다. 진단 결과 페이지(image_a3ae9e.jpg)의 흰색 박스들을 보면 경계가 아주 또렷하고 깔끔하죠.

3. 과감한 둥근 모서리 (Border-radius)

모든 하얀색 카드들의 모서리가 꽤 크게(약 16px ~ 24px) 둥글려져 있습니다. 이는 사용자에게 딱딱한 IT 서비스가 아니라 부드러운 뷰티 서비스를 이용하고 있다는 심리적인 안정감을 줍니다.

이미지 흐름을 분리해서 수정했습니다. 핵심은 이제 profile_image는 사용자 아바타 전용, generated_makeup_image는 진단 결과의 Gen AI 메이크업 이미지 전용으로만 사용됩니다. 마이페

이지/헤더/커뮤니티는 같은 useCurrentUser() 상태와 UserAvatar fallback을 쓰고, 결과 페이지

Gen AI 영역은 프로필 이미지나 톤 아이콘을 더 이상 fallback으로 쓰지 않습니다.

수정 파일

- Backend
 - Lumiere/back/accounts/models.py
 - Lumiere/back/accounts/serializers.py
 - Lumiere/back/accounts/profile_views.py
 - Lumiere/back/accounts/urls.py
 - Lumiere/back/accounts/migrations/0002_alter_user_profile_image.py
 - Lumiere/back/diagnosis/models.py
 - Lumiere/back/diagnosis/serializers.py
 - Lumiere/back/diagnosis/views.py
 - Lumiere/back/diagnosis/urls.py

- Lumiere/back/diagnosis/migrations/0003_diagnosisresult_generated_makeup_image_and_more.py
- Lumiere/back/community/serializers.py
- Lumiere/back/Lumiere/settings.py, Lumiere/back/Lumiere/urls.py
- Frontend
 - Lumiere/front/lumiere-frontend/src/components/AppHeader.vue
 - Lumiere/front/lumiere-frontend/src/views/accounts/MyPageView.vue
 - Lumiere/front/lumiere-frontend/src/components/accounts/ProfileEditModal.vue
 - Lumiere/front/lumiere-frontend/src/views/diagnosis/PersonalColorResultView.vue
 - Lumiere/front/lumiere-frontend/src/components/diagnosis/GeneratedMakeupImage.vue
 - Lumiere/front/lumiere-frontend/src/components/user/UserAvatar.vue
 - Lumiere/front/lumiere-frontend/src/composables/useCurrentUser.js
 - Lumiere/front/lumiere-frontend/src/services/userApi.js
 - Lumiere/front/lumiere-frontend/src/services/diagnosisApi.js
 - Lumiere/front/lumiere-frontend/src/constants/images.js
 - Lumiere/front/lumiere-frontend/public/images/default-profile.svg
 - 커뮤니티 아바타 사용처: PostItem, CommentThread, CommunityDetailContent, ToneLounge, CommunityView, CommunityDetailView

모델/API 변경

- User.profile_image: profiles/uploads/로 저장 경로 변경.
- DiagnosisResult에 분리 필드 추가:
 - uploaded_image

- generated_makeup_image
- /accounts/user/ 응답에 profile_image_url 추가.
- /accounts/user/update/ 추가: multipart/form-data로 프로필 이미지/닉네임/이메일 수정.
- /api/diagnosis/results/<id>/ 추가.
- 진단 결과 응답에 uploaded_image_url, generated_makeup_image_url 추가.
- 커뮤니티 게시물/댓글 serializer에 author_profile_image_url 추가.

프론트 상태 동기화

- useCurrentUser()를 추가해서 헤더와 마이페이지가 같은 로그인 사용자 상태를 봅니다.
- 프로필 저장 성공 시 saveCurrentUser()가 전역 currentUser를 갱신하므로 새로고침 없이 헤더/마이페이지 이미지가 같이 바뀝니다.

Fallback 방식

- 사용자 아바타 fallback은 한 곳에서 관리:
 - DEFAULT_PROFILE_IMAGE = "/images/default-profile.svg"
- 마이페이지/헤더/게시글/댓글/라운지 멤버는 UserAvatar를 사용합니다.
- 이미지 로드 실패 시에도 기본 프로필 이미지로 대체됩니다.

Gen AI 이미지 처리

- 결과 페이지의 Gen AI 영역은 GeneratedMakeupImage로 분리했습니다.
- 표시 우선순위:
 - generated_makeup_image_url 있으면 실제 생성 이미지 표시
 - 생성 중 상태 값이면 loading UI
 - 실패 상태 또는 이미지 로드 실패면 실패 안내 + 재시도 버튼
 - 없으면 명확한 empty state
- 프로필 이미지, 퍼스널컬러 톤 아이콘, 라운지 아이콘은 이 영역에 사용하지 않습니다.

검증 결과

- python manage.py makemigrations accounts diagnosis 성공
- python manage.py migrate 성공
- python manage.py check 성공
- python manage.py test community 성공, 6개 통과
- npm run build 성공

아직 실제 생성형 AI API는 연결되어 있지 않습니다. 현재는 generated_makeup_image 필드와 재

시도 버튼 자리만 준비되어 있고, 실제 생성/재생성 API가 붙으면 해당 이미지를 diagnosis/generated/에 저장해서 generated_makeup_image에 연결하면 됩니다.

views.py 가 다른 이유

▼ 방법론

백엔드 개발자들도 처음 장고(Django)를 배울 때 똑같은 의문을 가집니다. "어차피 복잡한 로직을 짤 거면, 내가 마음대로 컨트롤할 수 있는 FBV(함수형)로 다 짜는 게 더 좋은 거 아니야?" 라고요.

결론부터 말씀드리면, **FBV로 커뮤니티를 만들어도 절대 안 되는 것은 아닙니다.** 완벽하게 똑같이 동작하게 만들 수 있습니다.

하지만 커뮤니티 게시판 같은 기능을 FBV로 짜게 되면, 나중에 유지보수를 할 때 "개발자가 너무 고통스럽고 비효율적인 상황"을 맞이하게 됩니다. 그 이유를 3가지로 명확히 비교해 드릴게요.

1. '뻘하고 반복되는 코드'가 엄청나게 길어집니다.

커뮤니티 게시글(Post)을 다루려면 최소 5가지 기능이 필요합니다.

1. 게시글 전체 목록 보기 (GET)
2. 새 게시글 작성하기 (POST)
3. 게시글 하나 상세 보기 (GET)
4. 게시글 수정하기 (PUT/PATCH)
5. 게시글 삭제하기 (DELETE)

FBV로 짜면?

이 5가지 상황마다 데이터베이스에서 글을 찾고, 권한을 확인하고, 저장하고, 에러를 내뱉는 코드를 **일일이 다 손으로 써줘야 합니다**. 게시글 하나에만 수백 줄의 코드가 생기고, 댓글(Comment) 기능까지 만들면 또 똑같은 수백 줄의 코드를 복사해서 붙여넣어야 합니다.

CBV(ViewSet)로 짜면? `ModelViewSet` 을 상속받는 순간, 저 5가지 기능이 단 2줄의 코드 (`queryset`, `serializer_class`)만으로 알아서 완성됩니다. 코드가 1/10로 줄어드니 버그가 날 확률도 획기적으로 줄어듭니다.

2. ViewSet에서도 '복잡한 조작'은 얼마든지 가능합니다!

수빈님이 우려하신 "복잡한 조작이 안 되는 것 아닌가?" 하는 걱정은 안 하셔도 됩니다. 장고의 ViewSet은 기본 기능은 숨겨서 처리해주되, **필요한 부분만 쏙 빼서 개조할 수 있도록 완벽하게 설계되어 있습니다**.

올려주셨던 `views_1.py` 코드가 바로 그 증거입니다.

- 게시글 목록을 불러올 때 단순히 불러오지 않고, 카테고리(`category`), 제품 태그(`product`), 검색어(`q`)에 따라 필터링하는 복잡한 로직을 `get_queryset` 함수를 덮어쓰워서(Overriding) 커스텀했습니다.
- 조회수를 올리기 위해 `retrieve` 함수를 개조해서 게시글을 읽을 때마다 `view_count` 가 +1 되도록 만들었습니다.
- 기본 CRUD에 없는 '좋아요(Like)' 기능은 `@action` 을 달아서 별도의 함수로 깔끔하게 추가했습니다.

즉, 반복되는 노동은 프레임워크에 맡기고, 진짜 복잡하고 중요한 로직(검색, 좋아요 등)에만 집중할 수 있게 해주는 것이 ViewSet의 목적입니다.

3. URL 라우팅이 압도적으로 편해집니다.

- **FBV:** `urls.py` 에 `/posts/`, `/posts/<int:pk>/`, `/posts/create/` 등등 경로를 하나하나 다 적어줘야 합니다.
- **CBV (ViewSet):** 장고 프레임워크의 `DefaultRouter` 라는 마법의 도구를 쓰면, 단 한 줄만으로 게시판과 관련된 수십 개의 URL을 자동으로 생성해서 연결해 줍니다.

💡 실무 개발의 정석 (Golden Rule)

- 게시물, 댓글, 상품 정보, 회원 정보 수정 → 무조건 **CBV (ViewSet)** 사용 (표준화된 CRUD이기 때문)

- 로그인, 비밀번호 초기화, 결제 승인, 이메일 발송 → 무조건 **FBV** 사용 (표준 CRUD가 아닌, 특수한 로직의 집합이기 때문)

이 차이가 명확히 이해되셨나요? 백엔드의 뼈대가 이렇게 효율적으로 잘 잡혀 있다는 걸 확인했으니, 이제 프론트엔드로 넘어가서 **커뮤니티 글쓰기 전용 화면**

(`CommunityWriteView.vue`)의 UI 구조를 한번 짚을까요?

"FBV vs CBV, 언제 무엇을 써야 하는가?" 입니다.

결론부터 말씀드리면, "요즘은 무조건 **CBV**를 쓴다"는 것은 오해입니다. 두 가지 방식 모두 현재 실무에서 활발하게 쓰이고 있으며, 아키텍처를 설계할 때 각자의 명확한 역할이 있습니다.

보수적인 안정성을 최우선으로 생각하는 시스템 설계 관점에서, 이 두 가지를 언제 어떻게 나누어 써야 가장 견고한 백엔드가 완성되는지 명확한 기준을 세워드릴게요.

1. CBV (Class-Based View): "규격화된 자동화 공장"

CBV는 장고 프레임워크가 수년 동안 검증하고 다듬어 놓은 튼튼한 뼈대를 그대로 가져다 쓰는 방식입니다.

- **언제 써야 할까?** * 게시판(커뮤니티 글, 댓글), 상품 목록, 마이페이지 프로필 조회 등 **빠르고 반복적인 CRUD (생성, 조회, 수정, 삭제) 작업을 할 때.**
- **왜 써야 할까?**
 - 시스템에서 예기치 못한 버그나 장애가 발생하는 가장 큰 원인은 '개발자가 직접 짠 수많은 코드(휴먼 에러)' 때문입니다.
 - CBV를 쓰면 코드가 10분의 1로 줄어듭니다. 이미 전 세계 수백만 서비스에서 검증된 장고의 내부 로직을 그대로 타기 때문에, 에러가 날 확률이 극도로 낮아지고 시스템이 매우 안정적으로 동작합니다.

2. FBV (Function-Based View): "정밀한 수제 공방"

FBV는 위에서 아래로 코드가 흐르기 때문에, 중간에 어떤 일이 일어나는지 하나하나 명시적으로 통제할 수 있습니다.

- **언제 써야 할까?**
 - 로그인/회원가입, 비밀번호 초기화 메일 발송, 결제 처리, 외부 AI 모델 API 호출 (퍼스널 컬러 진단 등)처럼 복잡한 순서와 예외 처리가 필요한 작업.
- **왜 써야 할까?**

- CBV는 너무 많은 것들을 내부적으로 '알아서(Magic)' 처리해 버립니다. 만약 AI 사진 분석 로직을 CBV에 억지로 끼워 넣으려다 보면 프레임워크의 룰과 부딪혀 코드가 꼬이게 됩니다.
- 복잡한 비즈니스 로직에서는 "A가 성공하면 B를 하고, 실패하면 C를 해라"라는 흐름이 눈에 직관적으로 보여야 합니다. 그래야 장애가 발생했을 때 즉각적이고 안전하게 대처할 수 있습니다.

💡 Lumière 프로젝트 실전 적용 가이드

이 원칙을 수빈님이 지금 만들고 계신 프로젝트에 바로 대입해 볼까요?

- **✅ FBV로 짜야 하는 것 (정밀한 제어 필요)**
 - `login_view`, `signup_view`: 토큰을 발급하고 세션을 관리해야 함.
 - `ai_diagnosis_view`: 유저 사진을 받아서 -> 외부 AI 서버로 보내고 -> 결과를 기다렸다가 -> DB에 저장하는 복잡한 절차.
- **✅ CBV로 짜야 하는 것 (안정성과 표준화 필요)**
 - `PostViewSet`: 커뮤니티 게시글 전체 목록 불러오기, 새 글 쓰기.
 - `CommentViewSet`: 게시글에 달린 댓글들 불러오기, 삭제하기.

즉, "데이터베이스를 넣고 빼는 단순 작업은 가장 안전한 CBV에게 맡기고, 시스템의 중요한 핵심 로직이나 외부 연동 작업은 FBV로 개발자가 꼭 쥐고 통제한다"가 실무에서 가장 권장하는 아키텍처 분리 원칙입니다.

구분	FBV (함수 기반 뷰, Function-Based View)	CBV (클래스 기반 뷰, Class-Based View)
핵심 개념	개발자가 처음부터 끝까지 흐름을 직접 통제하는 수제 공방	검증된 프레임워크의 뼈대를 활용하는 자동화 공장
코드의 형태	<code>def</code> 키워드를 사용해 함수 단위로 작성	<code>class</code> 키워드를 사용해 객체 지향적으로 작성
주요 용도 (언제 쓰나요?)	복잡한 예외 처리나 순서가 중요한 특수 비즈니스 로직	표준화되고 반복적인 CRUD(생성/조회/수정/삭제) 로직

장점	- 코드의 실행 흐름이 위에서 아래로 직관적임 - 내가 원하는 대로 정밀한 제어와 변형이 가능함	- 코드가 획기적으로 짧아짐 (생산성 극대화) - 검증된 장고 로직을 타기 때문에 버그가 적음 - URL 라우팅 설정이 매우 간편함
단점	- 비슷한 기능(게시글, 댓글 등)을 만들 때마다 코드가 중복됨 - 코드가 길어지면 유지보수가 힘들어짐	- 내부적으로 어떻게 돌아가는지(Magic) 숨겨져 있어 흐름 파악이 어려움 - 정해진 규격을 크게 벗어나는 로직을 짜기 힘들
Lumière 적용 예시	(정밀 제어) - <code>login_view</code> : 토큰 발급 및 세션 관리 - <code>find_password_view</code> : 임시 비밀번호 메일 발송 - <code>ai_diagnosis_view</code> : 외부 AI 모델 연동 및 분석 대기	(표준 CRUD) - <code>PostViewSet</code> : 커뮤니티 게시글 목록 조회, 글쓰기 - <code>CommentViewSet</code> : 댓글 달기, 수정, 삭제

- **틀린 코드는 없습니다.** 두 방식 모두 실무에서 혼용해서 씁니다.
- **회원가입, 인증, 외부 API 연동**처럼 특수한 로직이 필요할 때는 수빈님이 아시는 대로 함수(FBV)로 짜는 것이 맞습니다 (`views.py`).
- 하지만 **커뮤니티 게시글, 댓글, 상품 목록**처럼 뻔한 생성/조회/수정/삭제가 필요한 기능은 코드를 획기적으로 줄이기 위해 클래스(ViewSet)로 짜는 것이 프론트엔드와 통신하기에도 훨씬 빠르고 안정적인 정석입니다 (`views_1.py`).

정리해야하는 문서 리스트

▼ 리스트

요약

요구사항 / 범위

요구사항 ID:

범위: P1 / P2 / IA-only / docs / chore

담당자:

리뷰어:

API 계약

엔드포인트:

요청 DTO:

응답 DTO:

오류 케이스:

소유권 검증 규칙:

관련 문서

요구사항:

API:

ERD:

테스트 계획:

화면:

체크리스트

요구사항 ID와 범위가 명확하다.

P2 또는 IA-only 동작을 활성 P1 동작으로 승격하지 않았다.

프론트/백엔드가 함께 바뀌는 경우 API 계약을 합의했다.

API 변경 시 API 문서를 갱신했다.

DB 변경 시 ERD/마이그레이션 문서를 갱신했다.

화면 또는 테스트 기준이 바뀐 경우 관련 문서를 갱신했다.

변경된 동작에 대한 테스트를 추가하거나 갱신했다.

secrets, 실제 토큰, 개인정보가 포함되지 않았는지 확인했다.

영향받는 앱 명령을 로컬에서 검증했다.

검증

스크린샷 / 참고

후속 작업

개발 기간/팀원 소개 도 넣어줘야 함

- [storyboard] 화면 구성 업데이트 한 내역 기록하기
 - > 메인 화면
 - >로그인/회원가입 화면 변경
 - > 퍼스널컬러 진단 화면 변경
 - > 제품별 컬러추천 화면 변경 (3단계 변경 됨)
 - >

▼ 추가 요청 리스트

[260614]

-personalcolor result , analysis result 명칭이 중복으로 겹칠 가능성이 있음

-result 라고 되어있는 view, component 명칭을 수정할 필요함 있음

- > 메인 페이지의 오늘의 인기 추천 제품에서
전체보기 누르면 어느 페이지로 이동하는가? -> 추천 페이지로 이동하기로 결정

AI 메이크오버 -> 이거는 요소(component)로 가져갈건가 아니면 view 로 가져갈건가
-> component 로 가져가는 게 맞을 듯 함

[새로 배운정보]

팝업/모달 디자인

캐러셀 슬라이더(가로 스크롤링)

flip 효과

난 네 실체 모습을 보고싶어

아이폰으로 우연히 찍힌 너의 일상을

아주 평범하고 잘못찍힌 듯한 스냅샷으로 보여줘

사진은 약간 모션블러가 있고 조명은 고르지 않고 자연스럽게 부탁해

저번기수까지는 커뮤니티 페이지가 따로 필요했는데 이제는 필요가 없을 수도 있다고 한다

HomeView.vue 파일의 세 군데(Template, Script, Style)를 아래 코드로 수정

GMS api

[260618]

마이페이지 -> 정보수정 버튼 활성화 (스토리보드에 업데이트 할 내용)

1. 같은 화면에서 변신하기 (v-if 사용, 컴포넌트 추가 X)

가장 쉽고 빠르게 구현할 수 있는 방법입니다. 평소에는 이름과 이메일 글자를 보여주다가, '정보 수정' 버튼을 누르면 그 자리가 <input> 입력창으로 스르륵 바뀌는 방식입니다.

장점: 새로운 파일을 만들 필요가 없습니다. 현재 마이페이지 컴포넌트 안에서 isEditing = true/false 변수 하나만으로 컨트롤할 수 있습니다.

단점: 수정할 항목(비밀번호, 프로필 이미지, 닉네임 등)이 너무 많으면 코드가 길어지고 복잡해집니다.

1. 모달(팝업) 창으로 띄우기 (새 컴포넌트 만들기 ○)

'정보 수정' 버튼을 누르면 화면 가운데에 작은 창(모달)이 떠서 정보를 수정하게 하는 방식입니다.

장점: 화면 이동 없이 깔끔하게 처리되며 UX(사용자 경험)가 좋습니다.

추천 구조: ProfileEditModal.vue 라는 이름으로 자식 컴포넌트를 하나 만들어서 마이페이지에 쏙 집어넣는 것이 좋습니다.

1. 아예 새로운 수정 페이지로 이동하기 (새 컴포넌트 만들기)

버튼을 누르면 /mypage/edit 같은 아예 새로운 주소의 화면으로 넘어가는 방식입니다. (네이버나 카카오톡 정보 수정과 비슷한 방식)

장점: 수정할 정보가 많고, 화면을 넓게 써야 할 때 가장 좋습니다.

추천 구조: ProfileEditView.vue 라는 화면 전체를 차지하는 뷰 컴포넌트를 하나 새로 만들어야 합니다.

 장고(Django) 백엔드도 준비가 필요합니다!

어떤 방식을 선택하든 프론트엔드 모양만 바뀔 뿐, 결국 수정한 데이터를 장고 서버로 보내서 DB를 업데이트해야 합니다.

이전에 내 정보를 조회(GET)하는 API(accounts/user/)를 만들었던 것처럼, 이번에는 내 정보를 수정(PATCH 또는 PUT)하는 API를 장고 쪽에 하나 뚫어주어야 합니다.

수빈님은 위 세 가지 방식(1. 현재 화면에서 전환, 2. 모달 팝업, 3. 아예 새 페이지로 이동) 중 어떤 방식이 지금 기획하신 디자인에 가장 잘 어울릴 것 같나요? 선택해 주시면 그 방식에 맞춘 Vue 코드와 장고 백엔드 코드를 연결하는 법을 안내해 드릴게요!

루미에르 비번 zhtc ylgc qntn uxcm

입술 상태도 봐주는 그런 것도 있었으면 좋겠다

입술

#8b3a4a

핵심 변경:

- 우측 사이드바 인기 제품 태그를 이미지 리스트에서 Pill Chip/Tag Cloud UI로 교체했습니다.
- 더 많은 태그 보기 클릭 시 /community?mode=tags로 중앙 영역이 전체 태그 탐색 화면으로 전환됩니다.
- 태그 단일/다중 필터를 URL query(tag, tags, mode)와 동기화했습니다.
- 태그 검색, 카테고리 필터, 정렬, 최대 3개 선택, 적용/초기화/개별 해제를 추가했습니다.
- 태그 적용 후 게시글이 없으면 전용 Empty State가 표시됩니다.
- /mypage, /community/posts/new 라우트에 인증 가드를 추가했습니다.

- 글쓰기 버튼/작성 제출/게시글 좋아요/상세 좋아요/댓글 관련 액션에 공통 로그인 가드를 붙였습니다.
- 401/403 응답 시 토큰 제거 후 로그인 페이지로 redirect되도록 처리했습니다.
- 로그인 성공 후 redirect query가 있으면 원래 목적지로 돌아갑니다.

추가한 주요 파일:

- /D:/python_study/copy/Lumiere/front/lumiere-frontend/src/components/community/PopularTagChip.vue
- /D:/python_study/copy/Lumiere/front/lumiere-frontend/src/components/community/PopularTagCloud.vue
- /D:/python_study/copy/Lumiere/front/lumiere-frontend/src/components/community/FullTagExplorer.vue
- /D:/python_study/copy/Lumiere/front/lumiere-frontend/src/components/community/SelectedTagBar.vue
- /D:/python_study/copy/Lumiere/front/lumiere-frontend/src/composables/useRequireLogin.js
- /D:/python_study/copy/Lumiere/front/lumiere-frontend/src/utils/auth.js
- /D:/python_study/copy/Lumiere/front/lumiere-frontend/src/utils/communityTags.js

수정한 중심 파일:

- /D:/python_study/copy/Lumiere/front/lumiere-frontend/src/views/community/CommunityView.vue
- /D:/python_study/copy/Lumiere/front/lumiere-frontend/src/components/community/CommunityRightSidebar.vue
- /D:/python_study/copy/Lumiere/front/lumiere-frontend/src/router/index.js
- /D:/python_study/copy/Lumiere/front/lumiere-frontend/src/components/accounts/LoginForm.vue

검증:

- npm run build 통과했습니다.

참고:

- 백엔드 Post 모델에는 실제 해시태그 저장 필드가 없어서, 태그 필터는 현재 프론트에서 hashtags mock, 게시물 제목/내용, products.brand/name/category 기준으로 매칭합니다.
- 백엔드 작성/좋아요 API는 이미 DRF 권한으로 인증이 걸려 있어 이번에는 프론트 가드와 401/403 UX 처리를 보강했습니다.

진단결과페이지

주요 변경:

- Lumiere/front/lumiere-frontend/src/views/diagnosis/PersonalColorResultView.vue:
결과 페이지를 컬러칩 중심 리포트 구조로 재구성했습니다. 제품 추천 영역은 본문에서 빼고 CTA로 낮췄습니다.
- Lumiere/front/lumiere-frontend/src/data/mockPersonalColorResults.js: 개발용 mock
결과 5종 추가
 - summer_cool_light
 - summer_cool_mute
 - winter_cool_clear
 - spring_warm_light
 - autumn_warm_deep
- Lumiere/front/lumiere-frontend/src/data/tonelImages.js: images/tones 실제 파일명
기준 프로필 이미지 매핑 정리. spring_warm_* 오타 수정, 실제 존재하는 summer_warm_ligh.png는 그대로 대응했습니다.
- Lumiere/front/lumiere-frontend/src/utils/diagnosisResultTransform.js:
API/mock
데이터를 결과 UI 표준 형태로 정규화하는 transform 추가.

- Lumiere/front/lumiere-frontend/src/utils/diagnosisMockStorage.js: 개발 환경 mock 저장용 localStorage 유틸 추가.
- Lumiere/front/lumiere-frontend/src/views/accounts/MyPageView.vue: 저장된 mock 진단 결과를 마이페이지에서 확인하고, toneKey 기반 프로필 이미지를 표시하도록 연결했습니다.
- Lumiere/front/lumiere-frontend/src/composables/useResolvedProfileImage.js: 헤더 프로필 이미지도 최신 진단 API 실패 시 개발용 저장 mock 결과를 fallback으로 사용할 수 있게 했습니다.
- UserAvatar, ProfileEditModal, SkinBalanceChart의 깨진 한글/접근성 문자열도 정상화했습니다.

검증:

- npm run build 통과했습니다.

Mock 진단 결과 접근 경로 및 개발용 계정 검증 계획

Summary

- /diagnosis/result?mock=... 경로를 추가해 개발 환경에서 로그인 없이 mock 진단 결과 화면을 바로 볼 수 있게 한다.
- 결과 저장하기, 마이페이지 반영, 헤더 프로필 반영은 로그인 상태에서만 가능하게 유지한다.
- 개발용 테스트 계정은 기존 seed_demo_diagnosis 관리 명령을 활용하되 기본값을 test@lumiere.dev / test1234!로 맞춘다.
- 운영 빌드에서는 mock query와 개발용 저장 fallback이 동작하지 않도록 import.meta.env.DEV 조건을 유지한다.

Key Changes

- 라우터에 path: '/diagnosis/result'를 추가하고 기존 PersonalColorResultView에 연결한다. 이 경로는 meta.requiresAuth를 붙이지 않는다.
- mock query 허용 목록은 다음 5개로 고정한다:
 - summer_cool_light
 - summer_cool_mute
 - winter_cool_clear
 - spring_warm_light
 - autumn_warm_deep
- 마이페이지에서 mock 결과 다시보기 링크는 /result?mock=... 대신 /diagnosis/result?mock=...로 이동하게 바꾼다.
- seed_demo_diagnosis의 기본 계정을 test@lumiere.dev / test1234!로 변경하고, 기존처럼 --email, --password 인자로 override 가능하게 둔다.
- 로그인은 기존 JWT 로그인 흐름을 그대로 사용한다:
 - POST /accounts/jwt-login/
 - username 값에는 이메일을 입력한다.
- mock 결과 저장은 개발 환경에서만 localStorage에 저장한다. 비로그인 상태에서 결과 저장하기 클릭 시 로그인 페이지로 redirect하고, 로그인 후 원래 mock 결과 URL로 돌아오게 한다.
- 헤더/마이페이지 프로필 이미지는 사용자가 직접 업로드한 이미지가 없을 때 저장된 mock 진단 결과의 toneKey 기반 이미지로 fallback한다.
- 커뮤니티 작성자 기본 프로필 이미지는 현재 API가 author_profile_image_url만 내려주므로, 이번 구현에서는 "업로드 이미지가 없으면 프론트 기본 아바타"를 유지한다. toneKey 기반 커뮤니티 작성자 이미지를 완전히 지원하려면 커뮤니티 post/

Test Plan

- 프론트 dev server 실행 후 아래 URL이 로그인 없이 열리는지 확인한다:
 - http://127.0.0.1:5174/diagnosis/result?mock=summer_cool_light
 - http://127.0.0.1:5174/diagnosis/result?mock=summer_cool_mute
 - http://127.0.0.1:5174/diagnosis/result?mock=winter_cool_clear
 - http://127.0.0.1:5174/diagnosis/result?mock=spring_warm_light
 - http://127.0.0.1:5174/diagnosis/result?mock=autumn_warm_deep
- 개발 계정 생성:
 - `python manage.py seed_demo_diagnosis`
 - 로그인 계정: test@lumiere.dev / test1234!
- 로그인 후 mock 결과에서 결과 저장하기를 누르고, /mypage에서 저장된 진단 결과와 toneKey 기반 프로필 이미지가 보이는지 확인한다.
- 헤더 우측 프로필 이미지가 사용자가 업로드한 이미지가 없을 때 저장된 mock 결과 tone 이미지로 바뀌는지 확인한다.
- `npm run build`로 production 빌드가 통과하는지 확인한다.
- production 빌드에서는 ?mock= query가 mock 데이터를 강제로 표시하지 않는지 확인한다.

Assumptions

- ERR_CONNECTION_REFUSED는 코드 라우팅 문제가 아니라 Vite dev server가 실행 중이지 않은 상태로 판단한다. 구현 후에는 서버를 실제로 계속 실행한 상태에서 URL을 확인해야 한다.
- 개발용 계정은 mock auth가 아니라 실제 백엔드 seed user + JWT 로그인으로 검증한다.
- 커뮤니티 작성자 tone 프로필 자동 반영은 API 응답 확장이 필요하므로 이번 변경에서는 확인 가능 여부를 명시하고 후속 범위로 둔다.